basics/06/app/models/project.rb

```
class Project
  attr_accessor :tasks

  def initialize
    @tasks = []
  end

  def incomplete_tasks
    tasks.reject(&:complete?)
  end

  def done?
    incomplete_tasks.empty?
  end

  def total_size
    tasks.sum(&:size)
  end

  def remaining_size
    incomplete_tasks.sum(&:size)
  end
end
```

This doesn't make the code shorter, but it does wrap a slightly opaque functional call containing a negative condition in a method with a semantically meaningful name. And if the definition of completeness changes, you only have to change one location.

There is still potential duplication in the Project class—two methods that call sum(&:size) on a list of tasks. I don't have an obvious place to put that method, though, short of creating a TaskList class. I don't see creating a TaskList as a simplification at this time, so we'll hold off. (A reviewer suggested this might also lead us to question whether the Project class is actually the correct abstraction and if TaskList might be better. Again, I think it's too early to tell.)

The First Date

You've got part of your Project API down; now you need to use that to calculate a projected completion date. The requirement is to calculate the project's end date based on the number of tasks finished in the last three weeks. I'll appropriate the agile term "velocity" to describe the rate of task completion. To make this work, you need to distinguish between tasks that concluded in the last three weeks and tasks that did not.

That means you have to deal with dates.

I'm sorry.